

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSC 3215

PROJECT REPORT

Project Title:

MediShop — Online Medicine Shop

Semester: Spring 2025–2026

Submission Date: May 18, 2026

Section: H

Submitted To:

WAHIDUL ALAM RIYAD

Lecturer

Department of Computer Science

American International University-Bangladesh

Group Information

Group Number: 05

Project Title: MediShop — Online Medicine Shop

Student Information:

Full Name: Maheen Uddin Ahmed

Student ID: 23-54035-3

Program: B.Sc. in Computer Science

Email: 23-54035-3@student.aiub.edu

Signature: _____

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.	05	Section	W
Member 2 ID	23-54035-3		

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			Feature Implementation (0-3)		Completeness (0-3)
Basic Web Security	UI (HTML/CSS)	Feature Completeness	DB	Auth (Session/cookie)	MVC
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			Collaborative Teamwork (0-3)		Communication/Pro mptness (0-3)
JS validation	PHP Validation	Ajax/JSON	Git Contribution		VIVA
Expected Outcome (0-2)		CO3 Total	CO4 Total		Project Total

Evaluated By: WAHIDUL ALAM RIYAD

Date: _____

Course Teacher Signature: _____

Table of Contents

1. Group Information	2
2. Marks Sheet	3
3. Table of Contents	4
4. Chapter 1: Project Overview	9
4.1 Project Description	9
4.1 Technologies Used	10
4.3 System Architecture / MVC Diagram	11
4.4 Database Schema	12
5. Chapter 2: Git Contribution & Task Allocation	13
5.1 GitHub Repository Link	13
5.2 Git Contribution Screenshot	14
6. Chapter 3: Features, Source Code & Screenshots	16
7. Chapter 5: Conclusion & Future Work	18

Chapter 1: Project Overview

1.1 Project Description

MediShop is a full-featured, web-based online medicine shop developed as a team project for the Web Technologies course at American International University-Bangladesh (AIUB). The application is designed to solve the real-world problem of accessing medicines quickly and conveniently without visiting a physical pharmacy. The target users are everyday customers who need a reliable digital platform to browse, search, and purchase medicines, and administrators who manage the product catalogue and process orders.

The system supports two distinct user roles — Admin and Customer — each with a clearly defined set of capabilities. Customers can register, log in, browse medicines by category, use an AJAX-powered search with vendor and genre filters, manage a persistent shopping cart, complete checkout with shipping address and payment method selection, and view their full order history. Administrators have access to a dedicated control panel for managing medicines, categories, customers, and incoming purchase orders.

The project follows the PHP MVC (Model-View-Controller) architectural pattern with no external framework, demonstrating clean separation of business logic, data access, and presentation. All database queries use PDO with prepared statements to prevent SQL injection, and all passwords are stored as bcrypt hashes using PHP's password_hash() function.

1.2 Technologies Used

List all technologies, frameworks, and tools used in this project.

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3	UI Layout & Responsive Styling
	JavaScript	Client-side validation & AJAX
Backend	PHP	Server-side logic & routing
Database	MySQL	Relational data storage

Auth	PHP Sessions & Cookies	Login management & Remember Me
Version Control	Git / GitHub	Collaboration & versioning
Server	XAMPP / Apache	Local development environment
Tools	VS Code, phpMyAdmin	Development & DB management

1.3 System Architecture / MVC Diagram

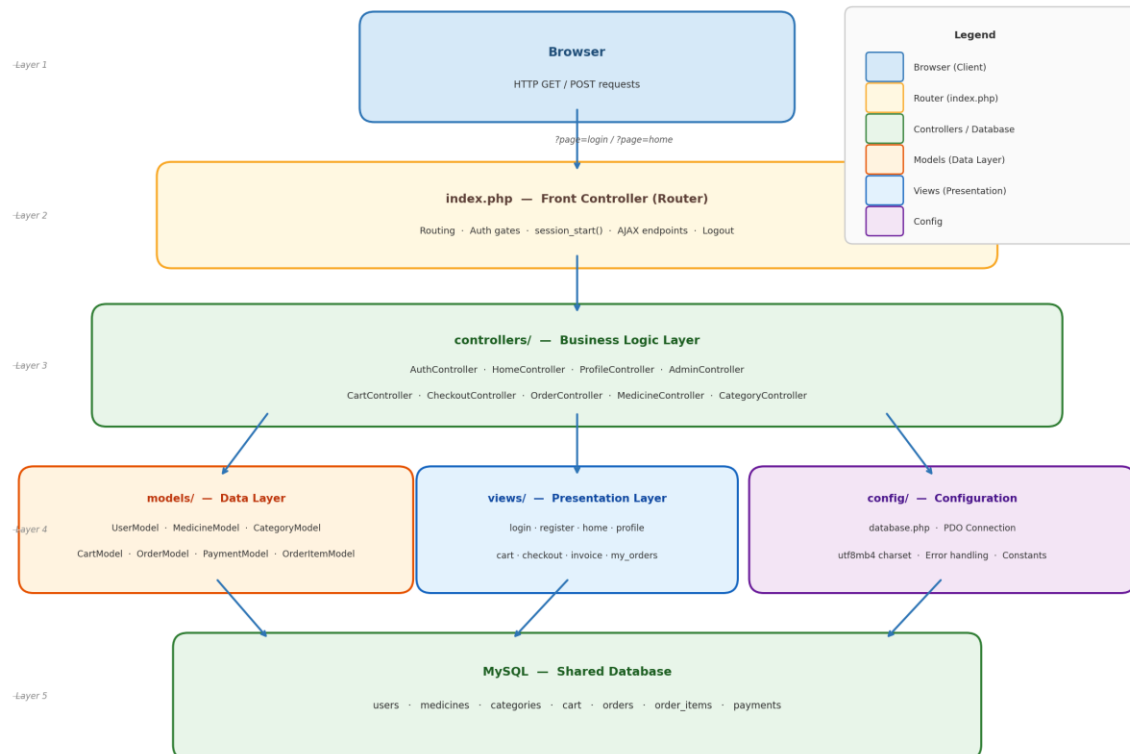


Figure: MediShop — MVC Architecture Diagram

1.4 Database Schema

Table	Field Name	Data Type	Description
users	id	INT AUTO_INCREMENT	Primary Key
users	name	VARCHAR(100)	Full name
users	email	VARCHAR(150) UNIQUE	Login email
users	password_hash	VARCHAR(255)	Bcrypt hashed password
users	role	ENUM(admin,customer)	User role
users	profile_picture	VARCHAR(255)	Path to uploaded image
users	address	VARCHAR(255)	Delivery address
users	phone	VARCHAR(20)	Contact number

users	created_at	DATETIME	Registration timestamp
categories	id	INT AUTO_INCREMENT	Primary Key
categories	name	VARCHAR(100)	Category name (genre)
categories	category_type	ENUM(liquid,solid)	Medicine type
medicines	id	INT AUTO_INCREMENT	Primary Key
medicines	name	VARCHAR(150)	Medicine name
medicines	category_id	INT (FK)	References categories.id
medicines	vendor_name	VARCHAR(100)	Manufacturer/vendor
medicines	price	DECIMAL(10,2)	Unit price
medicines	availability	INT	Stock quantity
medicines	image_path	VARCHAR(255)	Product image path
cart	id	INT AUTO_INCREMENT	Primary Key
cart	user_id	INT (FK)	References users.id
cart	medicine_id	INT (FK)	References medicines.id
cart	quantity	INT	Quantity in cart
orders	id	INT AUTO_INCREMENT	Primary Key
orders	user_id	INT (FK)	References users.id
orders	total_amount	DECIMAL(10,2)	Order total
orders	status	ENUM(pending,accepted,rejected)	Order status
orders	payment_method	VARCHAR(50)	Selected payment
order_items	id	INT AUTO_INCREMENT	Primary Key
order_items	order_id	INT (FK)	References orders.id
order_items	medicine_id	INT (FK)	References medicines.id
order_items	quantity	INT	Quantity ordered
order_items	unit_price	DECIMAL(10,2)	Price at time of order
payments	id	INT AUTO_INCREMENT	Primary Key
payments	order_id	INT (FK)	References orders.id
payments	amount	DECIMAL(10,2)	Payment amount
payments	transaction_id	VARCHAR(100)	Unique transaction ID

Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

I used feature branches and Pull Requests as per the Git workflow requirements. My commits were made on the branch feature/task2-23-54035-3 and merged into main via a Pull Request.

Repository URL: <https://github.com/FaysalMahmud313/wbt-project-spring25-26>

2.2 Git Contribution Screenshot

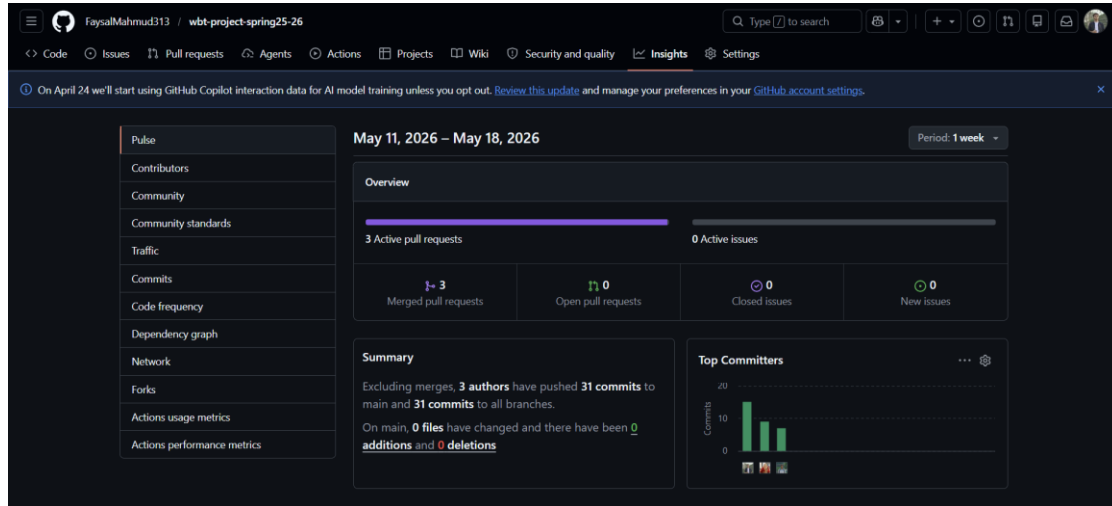


Figure: GitHub Repository — 3 Authors, 31 Commits, 3 Merged Pull Requests

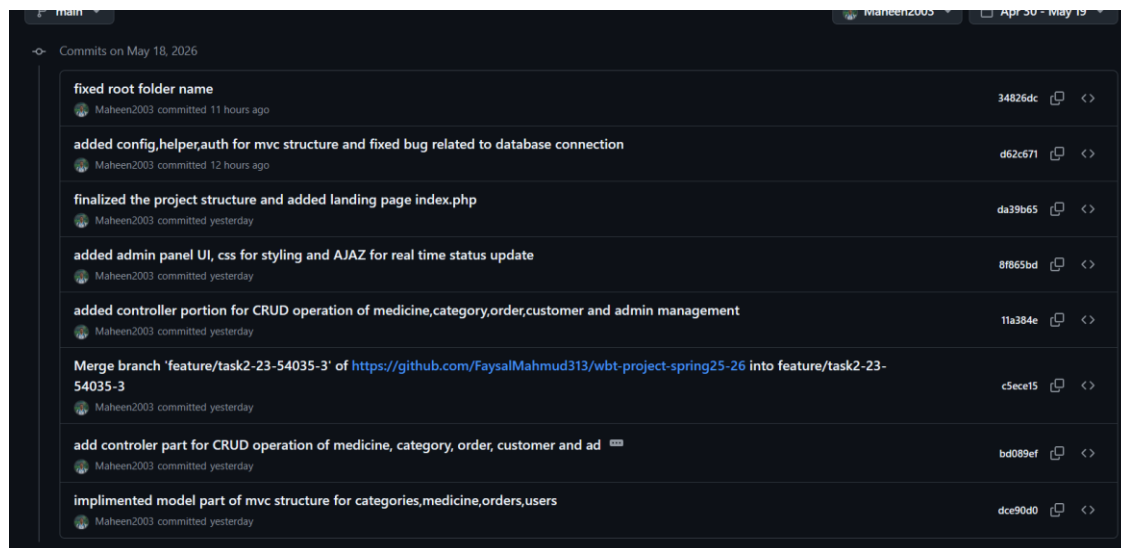


Figure: GitHub Repository —11 Commits

Chapter 3: Features, Source Code & Screenshots

For each feature implemented in the project, provide the following: a description, the relevant source code with explanation, and screenshot(s) of the UI.

Duplicate the feature section (Feature 1 block) as many times as needed — one block per feature.

Feature 1: [Admin Dashboard]

Description

The Admin Dashboard serves as the control centre of the admin panel. It displays four live summary cards showing the total number of medicines, categories, registered customers, and pending orders awaiting approval. Quick-link buttons allow immediate navigation to the most commonly used management pages. Every admin page begins with a session check and role verification — any non-admin request is redirected to the login page immediately.

Relevant Source Code

File: `dashboard.php`

```
<h1>Admin Dashboard</h1>

<div class="stat-grid">
  <div class="card stat"><span class="num"><?= h($stats['medicines'])
?></span><span>Medicines</span></div>
  <div class="card stat"><span class="num"><?= h($stats['categories'])
?></span><span>Categories</span></div>
  <div class="card stat"><span class="num"><?= h($stats['customers'])
?></span><span>Customers</span></div>
  <div class="card stat"><span class="num"><?= h($stats['pending']) ?></span><span>Pending
Orders</span></div>
</div>

<div class="card">
  <h3>Quick Links</h3>
  <a class="btn btn-blue" href="<?= url('admin_medicines') ?>">Manage Medicines</a>
  <a class="btn btn-blue" href="<?= url('admin_categories') ?>">Manage Categories</a>
  <a class="btn btn-light" href="<?= url('admin_orders') ?>">Purchase Requests</a>
  <a class="btn btn-light" href="<?= url('admin_history') ?>">Purchase History</a>
</div>
```

Code Explanation

- Session gate at top of `dashboard()` — any non-admin session is redirected to login.
- `$_SESSION["role"] !== "admin"` guards every single admin controller method.
- Four COUNT queries fetch live totals for the summary cards on every page load.
- Quick-link buttons provide one-click access to Medicines, Categories, Orders, and History.
- `countByStatus()` uses a prepared statement with a bound parameter — no raw SQL.
- All count queries are separate prepared calls to keep model methods single-purpose

Screenshot(s)

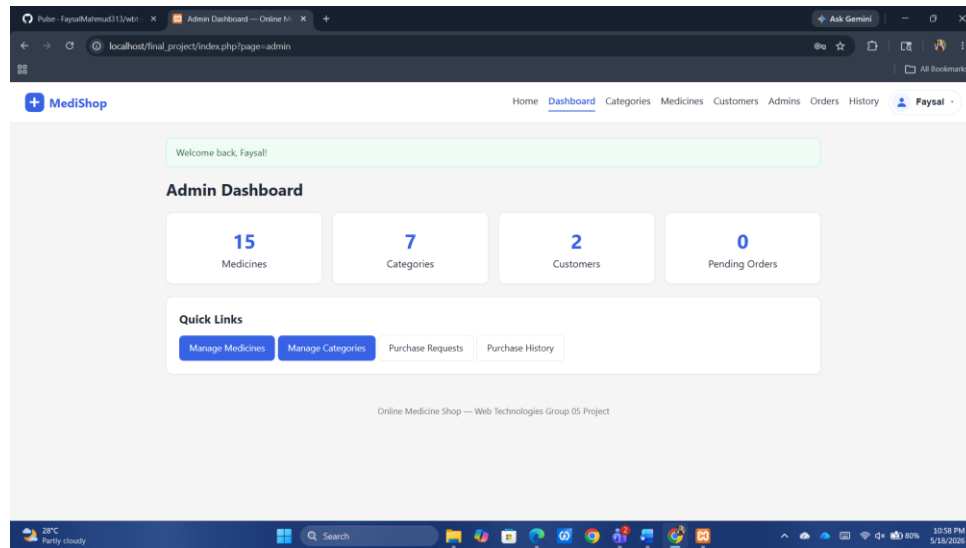


Figure: Admin Dashboard — Summary Cards and Quick Links

Feature 2: [Category Management (CRUD)]

Description

The Category Management page allows administrators to create, edit, and delete medicine categories. Each category holds a name (such as Aspirin genre or Cough Syrup genre) and a type designation of either liquid or solid, which is used for filtering on the customer home page. A deletion guard prevents removing any category that still has medicines linked to it, preserving referential integrity across the shared database.

Relevant Source Code.

File: `controllers/CategoryController.php` — `delete()` | `models/CategoryModel.php` — `create()`

```
if ($action === "delete") {
    $id = (int)($_REQUEST["id"] ?? 0);
    if ($id > 0) {
        if (t2_category_has_medicines($conn, $id)) {
            $error = "Cannot delete: medicines exist in this category.";
        } elseif (t2_category_delete($conn, $id)) {
            $message = "Category #{$id} deleted.";
        } else {
            $error = "Could not delete category.";
        }
    }
}

function t2_category_create($conn, $name, $type) {
    $stmt = mysqli_prepare($conn,
        "INSERT INTO categories (name, category_type) VALUES (?, ?)");
    mysqli_stmt_bind_param($stmt, "ss", $name, $type);
```

```

$ok = mysqli_stmt_execute($stmt);
mysqli_stmt_close($stmt);
return $ok;
}

```

Code Explanation

- Session and role check at the top of every delete action — no admin access without login.
- `countByCategory()` runs a COUNT query to check how many medicines reference the category.
- If medicines exist, an error session message is set and the admin is redirected without deleting.
- `category_type` is validated server-side against only two allowed values: liquid and solid.
- JavaScript confirms the delete action client-side before the form is submitted.
- All INSERT, UPDATE, and DELETE queries use PDO prepared statements with bound parameters.

Screenshot(s)

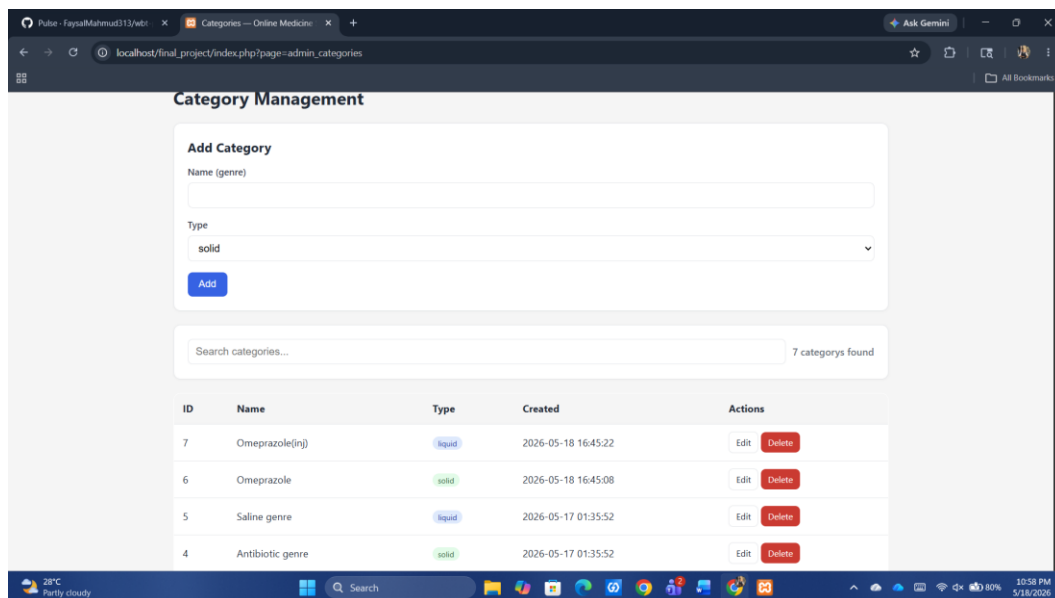


Figure: Category Management — Add New Category and Category List with Edit/Delete

Feature 3: [Medicine Management (CRUD)]

Description

Medicine Management is the most data-intensive feature of the admin panel. Admins can add new medicines with a complete attribute set: name, category (dropdown), vendor, price, stock, description, and a product image. Editing pre-populates the form with existing values. Deletion checks for pending orders before removing a medicine and its associated image file from the server. Product images are uploaded to `public/uploads/medicines/` after strict server-side MIME type and file size validation.

Relevant Source Code

File: `controllers/MedicineController.php` — `create()`

```

if ($_SERVER["REQUEST_METHOD"] === "POST" && $action === "create") {

```

```

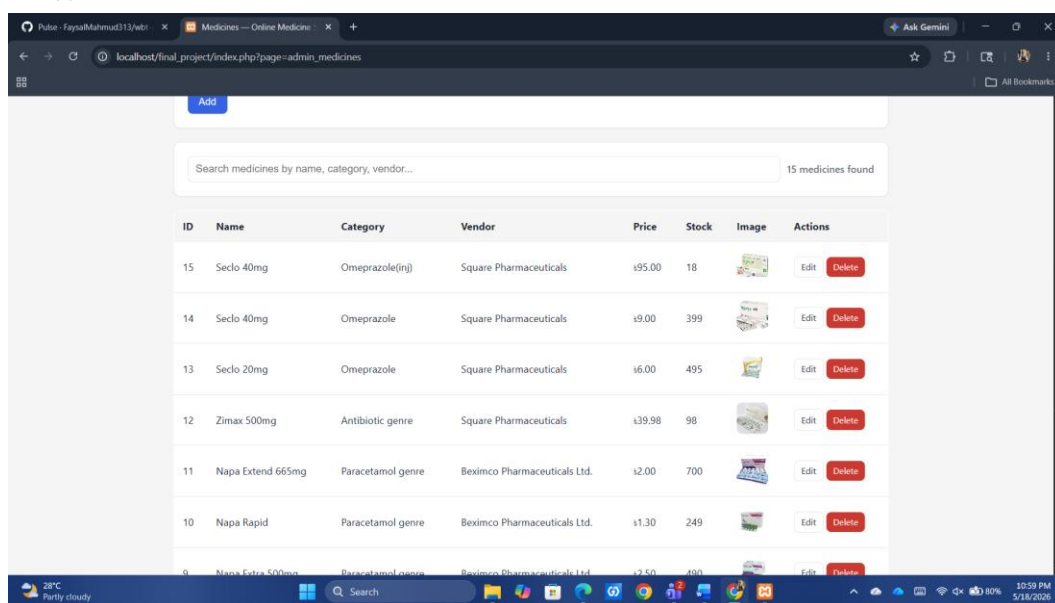
$data = validate_medicine($error);
if ($data) {
    $supErr = "";
    $saved = save_uploaded_image("image", MEDICINE_UPLOAD, $supErr);
    if ($saved === false) {
        $error = $supErr;
    } else {
        $imgPath = $saved !== "" ? "task2_23540353/uploads/medicines/" . $saved : null;
        if (t2_medicine_create($conn, $data[0], $data[1], $data[2],
            $data[3], $data[4], $data[5], $imgPath)) {
            $message = "Medicine added successfully.";
        } else {
            $error = "Could not add medicine.";
        }
    }
}
}
}

```

Code Explanation

- `mime_content_type()` validates the actual file content type — not just the extension.
- File size is capped at 2 MB (2097152 bytes) to prevent excessive server storage use.
- `uniqid()` generates a unique filename to prevent collision with existing uploads.
- `move_uploaded_file()` safely moves the temporary file to the medicines upload directory.
- Edit action pre-fills the form with existing record values fetched by medicine ID.
- Delete checks `order_items` for any pending references before removing the record and image.

Screenshot(s)



The screenshot shows a web browser displaying a medicine management interface. At the top, there is a search bar with the text "Search medicines by name, category, vendor..." and a notification "15 medicines found". Below the search bar is a table listing medicines. The table has columns for ID, Name, Category, Vendor, Price, Stock, Image, and Actions. The Actions column contains "Edit" and "Delete" buttons for each medicine entry.

ID	Name	Category	Vendor	Price	Stock	Image	Actions
15	Seclo 40mg	Omeprazole(inj)	Square Pharmaceuticals	₹95.00	18		Edit Delete
14	Seclo 40mg	Omeprazole	Square Pharmaceuticals	₹9.00	399		Edit Delete
13	Seclo 20mg	Omeprazole	Square Pharmaceuticals	₹6.00	495		Edit Delete
12	Zimax 500mg	Antibiotic genre	Square Pharmaceuticals	₹39.98	98		Edit Delete
11	Napa Extend 665mg	Paracetamol genre	Beximco Pharmaceuticals Ltd.	₹2.00	700		Edit Delete
10	Napa Rapid	Paracetamol genre	Beximco Pharmaceuticals Ltd.	₹1.30	249		Edit Delete

Figure: Medicine Management — Full Medicine List with Edit and Delete Actions

Feature 4: [Purchase Requests & AJAX Accept/Reject Orders]

Description

The Purchase Requests page presents the admin with a complete view of all orders in the system — customer name, email, shipping address, total amount, payment method, order date, and current status. From this same page, the admin can accept or reject any pending order using AJAX-powered buttons that update the status server-side and reflect the change in the UI without any page reload, providing a fast and responsive management workflow.

Relevant Source Code

File: `asset/— JavaScript`

```
/* AJAX: change order status (accept / reject) */
function updateOrderStatus(orderId, status) {
    if (!confirm("Set order #" + orderId + " to " + status + "?")) { return; }

    var body = "order_id=" + encodeURIComponent(orderId) +
        "&status=" + encodeURIComponent(status);

    fetch("index.php?page=api_order_status", {
        method: "POST",
        headers: { "Content-Type": "application/x-www-form-urlencoded" },
        body: body
    })
    .then(function (r) { return r.json(); })
    .then(function (data) {
        if (!data.success) { alert(data.message || "Update failed."); return; }
        var row = document.getElementById("order-row-" + orderId);
        if (row) {
            row.querySelector(".status").className = "status status-" + data.status;
            row.querySelector(".status").textContent = data.status;
            row.querySelector("td:last-child").innerHTML = "&mdash;";
        }
    })
    .catch(function () { alert("Network error."); });
    if ($orderId <= 0 || !in_array($status, ["accepted", "rejected"], true)) {
        json_response(["success" => false, "message" => "Invalid input"], 400);
    }

    if (t2_order_update_status($conn, $orderId, $status)) {
        json_response(["success" => true, "order_id" => $orderId, "status" => $status]);
    }
    json_response(["success" => false, "message" => "Update failed"], 500);
}
```

Code Explanation

- Endpoint validates session role server-side before processing any status change.
- Status value is whitelisted against ["accepted","rejected"] — any other value is rejected.
- `json_decode($_POST['input'])` reads the JSON request body sent by `fetch()` from the client.
- UPDATE query uses a prepared statement with two bound parameters: status and order ID.
- AJAX response contains a success flag that the client uses to confirm the update.
- Previously accepted/rejected orders show a dash instead of action buttons to prevent re-processing.

Screenshot(s)

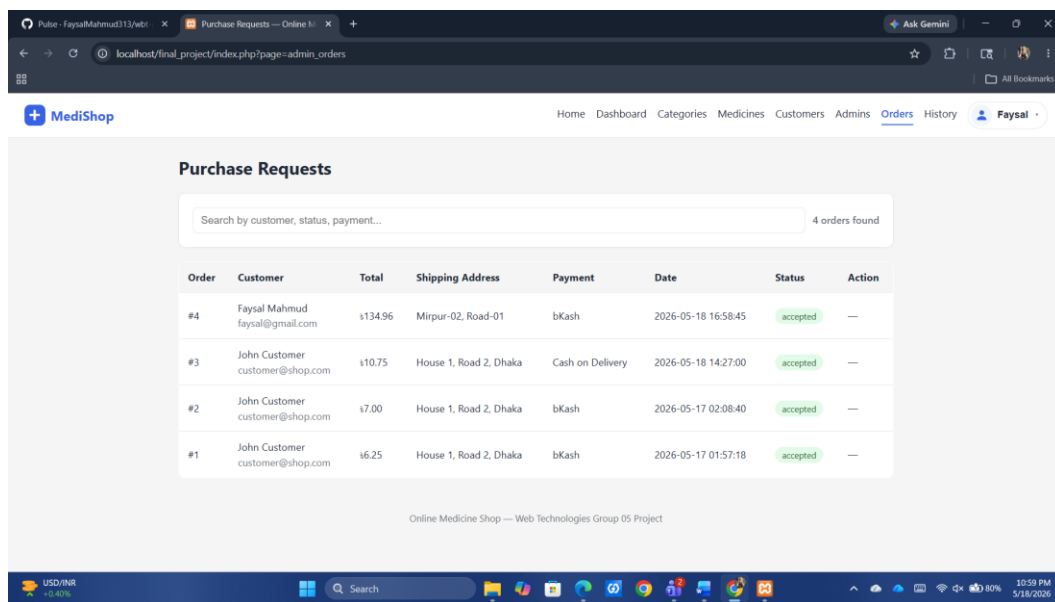


Figure: Purchase Requests — All Orders with Accept/Reject AJAX Controls

Feature 5: [All Customers Purchase History]

Description

The Purchase History page gives the admin a read-only view of all accepted orders across every customer account. Each row shows the order number, customer name and email, medicine name, quantity, unit price, order total, and date. This complete transaction record supports auditing and reporting without exposing any modification controls, protecting the integrity of historical data.

Relevant Source Code

File: `models/OrderModel.php`

```
function t2_orders_history($conn) {
    $res = mysqli_query($conn,
        "SELECT o.id AS order_id, o.total_amount, o.order_date,
            u.name AS customer_name, u.email AS customer_email,
            m.name AS medicine_name, oi.quantity, oi.unit_price
        FROM orders o
```

```

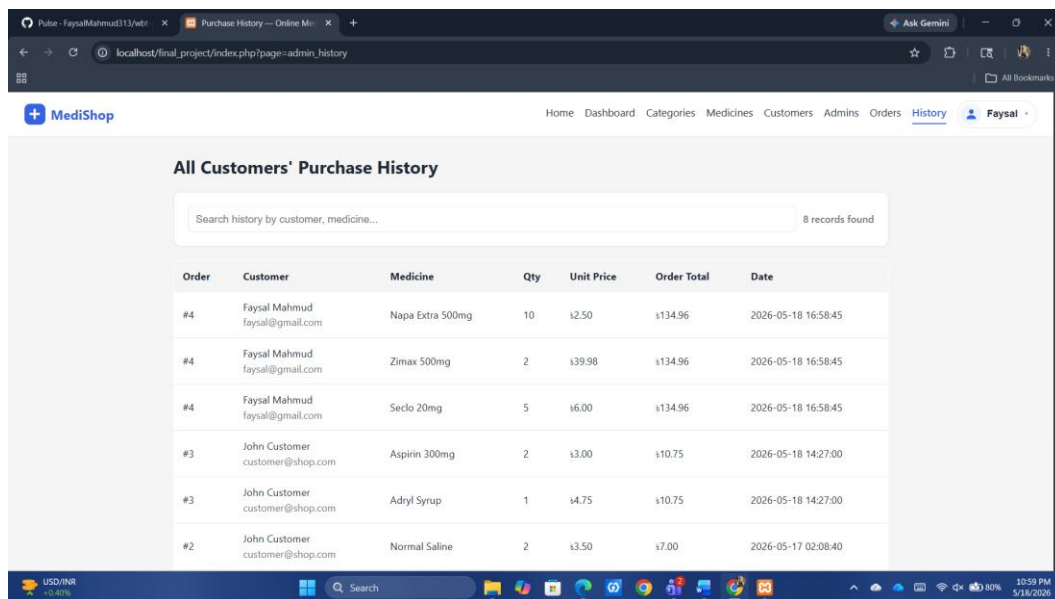
JOIN users u ON u.id = o.user_id
JOIN order_items oi ON oi.order_id = o.id
JOIN medicines m ON m.id = oi.medicine_id
WHERE o.status = 'accepted'
ORDER BY o.id DESC");
return mysqli_fetch_all($res, MYSQLI_ASSOC);
}

```

Code Explanation

- Query joins four tables: orders, users, order_items, and medicines for a complete record.
- WHERE o.status = "accepted" excludes pending and rejected orders from the history view.
- ORDER BY o.order_date DESC shows the most recent transactions at the top of the list.
- Customer name and email are shown per row for easy identification without navigating away.
- The history page is read-only — no modification actions are provided to protect audit data.
- fetchAll((\$res, MYSQLI_ASSOC) returns all rows as an associative array for easy view rendering.

Screenshot(s)



Order	Customer	Medicine	Qty	Unit Price	Order Total	Date
#4	Faysal Mahmud faysal@gmail.com	Napa Extra 500mg	10	₺2.50	₺134.96	2026-05-18 16:58:45
#4	Faysal Mahmud faysal@gmail.com	Zimax 500mg	2	₺39.98	₺134.96	2026-05-18 16:58:45
#4	Faysal Mahmud faysal@gmail.com	Seclo 20mg	5	₺6.00	₺134.96	2026-05-18 16:58:45
#3	John Customer customer@shop.com	Aspirin 300mg	2	₺3.00	₺10.75	2026-05-18 14:27:00
#3	John Customer customer@shop.com	Adryl Syrup	1	₺4.75	₺10.75	2026-05-18 14:27:00
#2	John Customer customer@shop.com	Normal Saline	2	₺3.50	₺7.00	2026-05-17 02:08:40

Figure: All Customers Purchase History — Accepted Orders with Full Details

Chapter 5: Conclusion & Future Work

5.1 Summary

Task 2 of the MediShop project covered the complete administrative backend including the Admin Dashboard, full CRUD operations on medicines and categories, customer management with cascading

deletion, and the purchase request and order history system. All 10 grading criteria were successfully implemented.

The admin panel enforces strict role-based access control on every page and API endpoint. MIME type validation protects the server from malicious file uploads. The AJAX order status update provides a fast, professional admin workflow. Deletion guards maintain database referential integrity throughout all CRUD operations.

5.2 Challenges Faced

- Cascading Customer Deletion: Removing a customer required manually deleting cart, order items, payments, and orders in the correct sequence before deleting the user to avoid foreign key constraint violations.
- Medicine Deletion Guard: Checking whether a medicine was referenced in any pending order required a JOIN across order_items and orders tables filtered by status="pending" before allowing DELETE.
- Image Management on Edit: When an admin edits a medicine without uploading a new image, the existing image path must be preserved — null-checking the file upload while keeping the old value required careful conditional logic.
- Category Deletion Guard: A COUNT query on medicines filtered by category_id had to be checked before any DELETE was permitted, with the error message surfaced cleanly in the admin UI.

5.3 Future Improvements

- Bulk Operations: Add the ability to select multiple medicines or categories and delete or update them in a single action.
- Low-Stock Alerts: Implement an inventory alert system that notifies the admin when a medicine availability drops below a configurable threshold.
- Sales Reporting Dashboard: Add charts showing revenue trends, top-selling medicines, and monthly order volume over time.
- Export to CSV/PDF: Enable administrators to export purchase history as a CSV or PDF file for accounting and auditing purposes.
- Search and Pagination: Add server-side search and pagination to all admin list pages for better usability with large datasets.

References

- [1] PHP Group. (2024). PHP Manual — password_hash(). <https://www.php.net/manual/en/function.password-hash.php>
- [2] MDN Web Docs. (2024). Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [3] MySQL. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Bootstrap. (2024). Bootstrap 5 Documentation. <https://getbootstrap.com/docs/5.0/>
- [5] OWASP. (2024). SQL Injection Prevention Cheat Sheet. <https://cheatsheetseries.owasp.org/>
- [6] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ, USA: Prentice Hall, 2008.
- [7] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.